

Dataset Design

Ezitech Engineering Framework — Case Study AI-005

Internship Performance Prediction & Risk Analytics Platform

Prepared by: Sadaf Riaz & Saira Ejaz

1. Why a Synthetic Dataset

Real Ezitech intern activity data was not available for this case study, so a synthetic dataset was generated instead (app/ml/generate_synthetic_data.py). The generator was built to be believable and learnable, not merely random: every intern's behaviour is driven by a hidden profile and trajectory rather than independent random numbers per field, so the resulting patterns resemble what a real cohort of interns would actually look like, and are the kind of pattern a model can legitimately learn.

2. Generation Parameters

Parameter	Value
Interns generated	800
Technology tracks	Laravel, MERN Stack, Artificial Intelligence, Flutter, UI/UX, DevOps
Batches	Batch-2026-A, Batch-2026-B, Batch-2026-C
Mentors generated	30, each with a max_capacity of 5–12 assigned interns
Internship length	90 days
Random seed	42 (fixed, for reproducible generation)

3. Hidden Profile and Trajectory Model

Each intern is assigned two hidden variables at generation time that jointly drive every other value recorded for them:

3.1 Profile — at_risk / average / strong

The profile sets the base range for attendance rate, task completion rate, commit activity, code review scores, and mentor ratings together, so a struggling intern shows multiple consistent low signals rather than one isolated random number — matching how a real struggling intern actually behaves.

3.2 Trajectory — improving / stable / declining

Layered on top of the profile, the trajectory causes attendance and task-completion probability to genuinely drift across the 90-day internship (not just review scores), using a drift range of +0.10 to +0.25 for improving, −0.05 to +0.05 for stable, and −0.25 to −0.10 for declining. This is what gives the Performance Trend and Learning Speed models a real, learnable time-based signal to detect, rather than only day-to-day noise around a constant average.

Trajectory probability itself depends on profile — an at_risk intern has a 55% chance of a declining trajectory versus 15% improving, while a strong intern has a 65% chance of improving versus 10% declining — and trajectory in turn adjusts the final dropout probability via a multiplier (1.4× for declining, 1.0× for stable, 0.6× for improving), applied on top of the profile's base dropout rate. This ties the two hidden variables together the same way real risk and real momentum would interact.

3.3 Realistic Overlap (Noise)

Dropout is drawn probabilistically from these adjusted rates (55% base for at_risk, 12% for average, 3% for strong, before the trajectory multiplier) rather than being a deterministic rule. This intentionally allows some at-risk interns to succeed and some strong interns to still drop out — avoiding a dataset with perfectly separable classes, which would produce a suspiciously perfect and practically meaningless model.

4. Final Dataset — Actual Generated Counts

Counts below are taken directly from the project's exported dataset (data_export.dump).

Table	Row Count
interns	800
mentors	30
attendance	28,222
tasks	9,692
github_activity	20,216
code_reviews	8,284
mentor_feedback	6,060
communication_activity	13,610

4.1 Intern Status Distribution

Status	Count
completed	478
active	165
dropped	157

4.2 Technology Distribution

Technology	Count
DevOps	152
Flutter	141
UI/UX	140
Artificial Intelligence	128
MERN Stack	123
Laravel	116

4.3 Batch Distribution

Batch	Count
Batch-2026-B	271
Batch-2026-C	268
Batch-2026-A	261

5. Database Schema

13 tables across the core schema (sql/schema.sql) and two later migrations:

Group	Tables
Core entities	mentors, interns
Activity/behavioural data	attendance, tasks, github_activity, code_reviews, mentor_feedback, communication_activity, weekly_performance
Model outputs	predictions, recommendations
Authentication (migration)	users — linked to intern_id or mentor_id for login
Student self-service (migration)	weekly_reports, project_submissions

6. Reproducibility

The generator's fixed random seed (42) means the same generation code always produces the same dataset. This was necessary in practice: with two people working on the same project, an unpinned/reference-date-dependent generator was found to silently produce different datasets depending on which day it was run — this was identified and fixed by pinning the seed and deriving all date-relative calculations from the data itself rather than the real-world calendar date, so both team members' databases stay identical and directly comparable.